

ONEDRAFT

CAD-AI — Aria Powered

Compliance & Regulatory Engine

Version 0.1

Status: Implementation Specification

Engine: Compliance & Regulatory Engine

Database: PostgreSQL

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

Execution: Python Services / Workers

Persistence: PostgreSQL + Object Storage

Job Queue: Redis

Geometry Source: Versioned Computational Project Model

1. COMPLIANCE ENGINE PRINCIPLE

The OneDraft Compliance & Regulatory Engine is responsible for determining whether a specific version of a project model satisfies the applicable regulatory requirements associated with the project jurisdiction.

The Compliance Engine does not create the architectural model.

It does not directly modify geometry.

It does not directly modify project elements.

It does not replace professional review or authority approval.

Its function is to establish a deterministic, traceable relationship between:

Jurisdiction

Regulatory Sources

Code Rules

Project Characteristics

Applicable Requirements

Project Model

Validation Results

Provenance

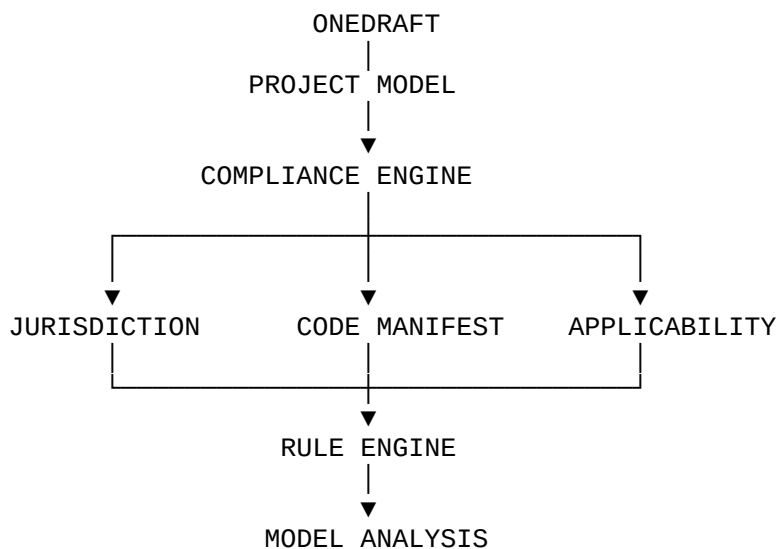
The fundamental computational chain is:

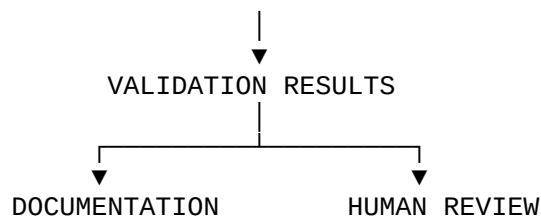
```
PROJECT
↓
JURISDICTION
↓
CODE SOURCES
↓
CODE MANIFEST
↓
APPLICABILITY RESOLUTION
↓
CODE RULE SET
↓
PROJECT CHARACTERISTICS
↓
MODEL ANALYSIS
↓
VALIDATION RESULTS
↓
DOCUMENTATION / REVIEW
```

2. COMPLIANCE ENGINE BOUNDARY

The Compliance Engine is a domain service positioned between the Project Model and the Validation Engine.

The architecture is:





The Compliance Engine therefore becomes the regulatory interpretation and analysis layer of OneDraft.

3. CORE DESIGN RULE

The Compliance Engine shall never treat a code source as merely a text document.

A regulatory source must be transformed into structured computational objects.

The system therefore separates:

```
graph TD; SD[SOURCE DOCUMENT] --> SV[SOURCE VERSION]; SV --> CR[CODE RULE]; CR --> AL[APPLICABILITY LOGIC]; AL --> VM[VERIFICATION METHOD]; VM --> MT[MODEL TEST]; MT --> R[RESULT];
```

This allows regulatory requirements to become machine-evaluable without losing their original provenance.

4. REGULATORY PROVENANCE PRINCIPLE

Every compliance determination must be traceable to its regulatory source.

A validation result must be capable of answering:

Which jurisdiction?

Which authority?

Which code source?

Which source version?

Which effective date?

Which rule?

Which applicability condition?

Which project revision?

Which model version?

Which objects were evaluated?

Which calculation was performed?

Which result was produced?

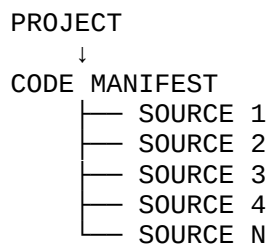
When was the determination produced?

No final compliance result should exist without a provenance chain.

5. IMMUTABLE CODE MANIFEST

The Code Manifest is the regulatory snapshot applicable to a project revision.

Conceptually:



A Code Manifest contains:

jurisdiction
authority
source identifiers
source versions
effective dates
retrieval timestamps
content hashes
applicable regulatory domains

Once a Code Manifest is attached to a finalized revision, it becomes immutable.

A later regulatory update creates a new manifest.

The previous manifest remains available for historical reconstruction.

6. REGULATORY TIME MODEL

Regulatory applicability is time-dependent.

The engine must distinguish:

publication date
effective date
retrieval date
project date
application date
permit date
revision date

A newer code source must not automatically overwrite the regulatory basis of an earlier project revision.

The compliance engine evaluates the regulatory environment associated with the applicable project timeline.

7. JURISDICTION MODEL

Jurisdiction resolution shall be hierarchical.

Conceptually:

COUNTRY
↓
PROVINCE / STATE
↓
MUNICIPALITY
↓
DISTRICT / AUTHORITY
↓
PROJECT

Additional regulatory authorities may apply independently.

Examples include:

building authority
fire authority
accessibility authority
planning authority
energy authority
environmental authority
utility authority
transportation authority

The engine must therefore support multiple simultaneous regulatory authorities.

8. JURISDICTION OBJECT

Conceptually:

Jurisdiction

contains:

country
province_state
municipality
district
authority
jurisdiction_code
effective_date
status

A project may have more than one jurisdictional relationship.

9. JURISDICTION RESOLUTION

The jurisdiction resolver receives:

project.address
project.jurisdiction
project.project_type
project.application_date

and produces:

Jurisdiction Resolution

containing:

primary_jurisdiction
secondary_jurisdictions
authorities
regulatory_domains
confidence
provenance

The resolution itself is recorded.

10. CODE SOURCE MODEL

A Code Source represents a regulatory publication or authoritative source.

The source may be:

national code
provincial code
state code
municipal bylaw
technical standard
authority publication
adopted standard
amendment
bulletin
regulatory interpretation

Each source receives a persistent UUID.

11. CODE SOURCE RECORD

The existing persistence layer defines:

`compliance.code_sources`

with:

```
id UUID PRIMARY KEY
jurisdiction VARCHAR(255) NOT NULL
authority VARCHAR(255)
title VARCHAR(512) NOT NULL
source_uri TEXT
version VARCHAR(128)
effective_date DATE
retrieved_at TIMESTAMPTZ
content_hash VARCHAR(128)
status VARCHAR(32) NOT NULL
```

The Compliance Engine extends this record conceptually through associated source artifacts and structured rule records.

12. SOURCE ARTIFACT

The source artifact is the preserved representation of the regulatory material used to create the computational rule set.

It may contain:

PDF
HTML
XML
JSON
structured publication
authority data
machine-readable standard

The source artifact shall be stored through the OneDraft object-storage subsystem where legally and technically permissible.

The database retains the provenance metadata.

13. SOURCE HASHING

Every imported source artifact should receive a cryptographic content hash.

Conceptually:

```
source_content
  ↓
SHA-256
  ↓
content_hash
```

The hash provides an integrity identifier for the regulatory source used by the engine.

14. CODE RULE OBJECT

A Code Rule is the computational representation of a regulatory requirement.

Conceptually:

Code Rule

contains:

```
rule_id
source_id
citation
rule_type
requirement
applicability
threshold
unit
exceptions
verification_method
```

The rule must preserve both:

human-readable requirement

and:

machine-readable evaluation logic

15. RULE TYPES

The initial rule taxonomy shall support:

NUMERIC
BOOLEAN
ENUMERATION
CLASSIFICATION
RELATIONAL
SPATIAL
GEOMETRIC
AREA
HEIGHT
WIDTH
DISTANCE
CLEARANCE
COUNT
RATIO
PERCENTAGE
ASSEMBLY
OCCUPANCY
EGRESS
ACCESSIBILITY
FIRE
STRUCTURAL
ENERGY
PARKING
SITE
DOCUMENTATION
COORDINATION

Additional types may be introduced without changing the core engine architecture.

16. RULE STRUCTURE

A computational rule may be represented conceptually as:

```
RULE
{
    identifier
    citation
    requirement
    applicability
    inputs
    operator
    threshold
    units
    exceptions
    verification_method
    severity
}
```

Example:

INPUT:
building.height

OPERATOR:
<=

THRESHOLD:
45

UNIT:
m

RESULT:
PASS / FAIL

17. APPLICABILITY ENGINE

A rule must first be determined to be applicable.

The engine therefore performs:

RULE
↓
APPLICABILITY TEST
↓
APPLICABLE / NOT APPLICABLE / UNDETERMINED

The engine must not report a rule violation merely because a rule exists.

It must first establish that the rule applies to the project.

18. APPLICABILITY INPUTS

Applicability may depend upon:

jurisdiction
building type
occupancy
construction type
building area
building height
number of storeys
site conditions
project scope
alteration type
new construction
existing building status
fire classification
occupant load
accessibility classification
energy classification

project date

These values are derived from the Project Model and project metadata.

19. APPLICABILITY STATES

Initial applicability states:

APPLICABLE
NOT_APPLICABLE
UNDETERMINED
CONDITIONAL
SUPERSEDED
EXEMPT

UNDETERMINED is a valid state and must not automatically become PASS.

20. COMPLIANCE DETERMINATION STATES

Validation results shall support:

PASS
FAIL
WARNING
REVIEW_REQUIRED
NOT_APPLICABLE
UNDETERMINED
ERROR

The distinction between FAIL and REVIEW_REQUIRED is critical.

A rule that cannot be deterministically evaluated must not be represented as a failed rule unless the applicable logic establishes failure.

21. SEVERITY MODEL

Initial severity levels:

INFO
LOW
MEDIUM
HIGH
CRITICAL

Severity is independent from result status.

For example:

```
status = REVIEW_REQUIRED  
severity = HIGH
```

is valid.

22. COMPLIANCE DOMAINS

The initial engine shall support:

```
BUILDING  
FIRE  
ACCESSIBILITY  
PLANNING  
ZONING  
ENERGY  
SITE  
STRUCTURAL  
EGRESS  
PARKING  
LIFE_SAFETY  
DOCUMENTATION  
COORDINATION
```

The architecture must allow additional domains.

23. PROJECT CHARACTERISTICS

Before rules are evaluated, the Compliance Engine derives a normalized set of project characteristics.

Conceptually:

Project Characteristics

may contain:

```
building_area  
building_height  
storey_count  
occupancy  
construction_type  
fire_rating  
occupant_load  
egress_count  
egress_distance  
accessible_routes  
parking_count  
setback_distances
```

site_area
window_area
wall_area
floor_area
energy_properties

These characteristics are computed from the Project Model.

24. CHARACTERISTIC DERIVATION

The engine shall distinguish between:

stored characteristics

and:

derived characteristics

For example:

Level areas
↓
Building gross area
↓
Building area characteristic

or:

Space occupancy
+
Space occupant load
↓
Building occupant load

Derived characteristics must preserve their calculation provenance.

25. CHARACTERISTIC PROVENANCE

Every derived characteristic should identify:

project_id
revision_id
model_version_id
source_objects
calculation_method
calculation_timestamp

This permits reconstruction of how a compliance input was produced.

26. MODEL SNAPSHOT

Compliance analysis operates against an immutable model snapshot.

Conceptually:

```
MODEL VERSION 17
      ↓
COMPLIANCE ANALYSIS
```

The engine must never evaluate a moving target.

If the project changes:

```
MODEL VERSION 17
      ↓
MODEL VERSION 18
```

the compliance analysis associated with Version 17 remains associated with Version 17.

27. COMPLIANCE RUN

A compliance run is a complete execution of the regulatory analysis against a model version and Code Manifest.

Conceptually:

ComplianceRun

contains:

```
project_id
revision_id
model_version_id
code_manifest_id
started_at
completed_at
status
rule_count
result_count
summary
engine_version
```

28. COMPLIANCE RUN STATES

Initial states:

```
QUEUED
RUNNING
```

RESOLVING
ANALYZING
COMPLETED
FAILED
CANCELLED
STALE

A run becomes STALE when its model version is no longer current and the application requires a current compliance determination.

The historical run remains retained.

29. COMPLIANCE RESULTS

Compliance results are persistent records.

They shall identify:

validation_run_id
project_id
revision_id
rule_id
affected_objects
category
status
severity
description
source
created_at

The existing table:

validation.validation_results

becomes the principal persistence layer for these results.

30. RULE EXECUTION PIPELINE

The execution pipeline becomes:

```
LOAD PROJECT
  ↓
LOAD MODEL VERSION
  ↓
LOAD CODE MANIFEST
  ↓
LOAD CODE RULES
  ↓
RESOLVE APPLICABILITY
  ↓
```

```
DERIVE PROJECT CHARACTERISTICS
↓
EXECUTE RULES
↓
CREATE RESULTS
↓
CREATE PROVENANCE
↓
CREATE SUMMARY
↓
COMPLETE RUN
```

31. RULE EXECUTION IS READ-ONLY

The Compliance Engine shall evaluate the Project Model.

It shall not directly mutate:

```
elements
spaces
levels
assemblies
geometry
constraints
relationships
drawings
```

Compliance findings may produce recommendations or commands for Aria, but those commands must pass through the established Project Model command boundary.

32. ARIA COMPLIANCE INTERFACE

Aria may consume compliance results.

Conceptually:

```
Compliance Result
↓
Aria
↓
Design Proposal
↓
Command
↓
Project Model
↓
New Revision
↓
Compliance Re-run
```

This prevents the AI from bypassing the transactional model architecture.

33. COMPLIANCE RECOMMENDATIONS

A failed or review-required rule may produce a recommendation.

Conceptually:

```
Finding
↓
Recommendation
↓
Design Proposal
```

A recommendation is not itself a model mutation.

34. COMPLIANCE COMMAND BOUNDARY

The following pattern is mandatory:

```
COMPLIANCE
↓
FINDING
↓
RECOMMENDATION
↓
ARIA COMMAND
↓
COMMAND VALIDATION
↓
MODEL TRANSACTION
```

The Compliance Engine never executes a direct database mutation against the Project Model.

35. CODE MANIFEST CREATION

The manifest generation workflow is:

```
PROJECT ADDRESS
↓
JURISDICTION RESOLUTION
↓
AUTHORITY RESOLUTION
↓
SOURCE DISCOVERY
↓
SOURCE VERSION SELECTION
```

↓
EFFECTIVE-DATE CHECK
↓
SOURCE HASH
↓
CODE MANIFEST

The resulting manifest receives a UUID.

36. CODE MANIFEST IMMUTABILITY

Once used by a finalized revision:

```
code_manifest.status = FINAL
```

The manifest cannot be modified.

A regulatory change creates:

CODE MANIFEST N+1

rather than modifying:

CODE MANIFEST N

37. CODE MANIFEST STATUS

Initial statuses:

DRAFT
ACTIVE
FINAL
SUPERSEDED
ARCHIVED
INVALID

A FINAL manifest is immutable.

38. REGULATORY SOURCE STATUS

Initial source statuses:

DISCOVERED
RETRIEVED
VERIFIED
ACTIVE

SUPERSEDED
ARCHIVED
INVALID

The system must not silently treat an unverified source as authoritative.

39. SOURCE AUTHORITY

Each source records its authority context.

Conceptually:

```
authority
authority_type
jurisdiction
publication
version
effective_date
```

The engine uses authority metadata when constructing a Code Manifest.

40. REGULATORY CONFLICTS

Multiple regulatory sources may produce competing requirements.

The engine therefore requires a conflict-resolution layer.

Conceptually:

```
RULE A
+
RULE B
+
RULE C
  ↓
CONFLICT DETECTION
  ↓
AUTHORITY / APPLICABILITY ANALYSIS
  ↓
RESOLUTION
```

An unresolved conflict becomes:

REVIEW_REQUIRED

rather than being silently resolved by AI inference.

41. RULE PRIORITY

Rule priority may be determined from:

- authority
- jurisdiction
- effective date
- adoption status
- specificity
- amendment relationship
- exception relationship

The priority model must remain explicit.

The engine must never rely on undocumented ordering.

42. EXCEPTIONS

Rules may contain exceptions.

Conceptually:

```
RULE
├── GENERAL REQUIREMENT
└── EXCEPTION
    ├── APPLICABILITY CONDITION
```

An exception must be evaluated independently.

The engine must record:

- exception_id
- exception_condition
- evaluation_result
- provenance

43. THRESHOLD ENGINE

The threshold engine supports:

- <
- <=
- =
- >=
- >
- !=
- BETWEEN
- OUTSIDE
- IN
- NOT_IN

Example:

```
actual = 1.4 m  
required >= 1.2 m
```

Result:

PASS

44. UNIT NORMALIZATION

The Compliance Engine uses canonical internal units.

The OneDraft computational model uses:

millimetres

for internal linear dimensions.

Regulatory rules may use:

mm
cm
m
in
ft
ft-in

The compliance engine normalizes values before evaluation.

Example:

```
1.2 m  
↓  
1200 mm
```

The original regulatory unit remains available for provenance and reporting.

45. DIMENSIONAL VALIDATION

The engine must verify that a comparison is dimensionally valid.

For example:

distance vs distance
area vs area
volume vs volume
height vs height
count vs count
ratio vs ratio

Invalid dimensional comparisons become:

ERROR

rather than producing a numerical result.

46. SPATIAL RULE ENGINE

Certain regulations cannot be evaluated as simple scalar values.

The engine therefore supports spatial predicates.

Initial predicates:

WITHIN
CONTAINS
INTERSECTS
TOUCHES
DISTANCE_FROM
OVERLAPS
SEPARATED_FROM
ALIGNED_WITH
ACCESSIBLE_FROM
CONNECTED_TO

Spatial evaluations operate against the Geometry Engine.

47. SPATIAL ANALYSIS PIPELINE

```
CODE RULE
  ↓
SPATIAL REQUIREMENT
  ↓
MODEL OBJECT SELECTION
  ↓
GEOMETRY QUERY
  ↓
GEOMETRY RESULT
  ↓
RULE EVALUATION
  ↓
COMPLIANCE RESULT
```

The Compliance Engine consumes geometry results rather than directly owning geometric computation.

48. EGRESS ENGINE

The initial egress subsystem evaluates:

- occupant load
- required exits
- available exits
- exit capacity
- travel distance
- common path
- exit separation
- exit access
- exit discharge

The precise rule parameters originate from the applicable Code Manifest.

49. ACCESSIBILITY ENGINE

The accessibility subsystem evaluates applicable requirements such as:

- accessible routes
- door clearances
- turning areas
- corridor widths
- ramps
- stairs
- washrooms
- parking
- accessible entrances
- maneuvering clearances

Each test is linked to the applicable rule.

50. FIRE AND LIFE-SAFETY ENGINE

The fire/life-safety subsystem evaluates applicable parameters including:

- occupancy classification
- fire separations
- fire-resistance ratings
- exit requirements
- fire access
- compartmentation
- opening protection
- assembly performance

The engine does not assume that any individual parameter is universally applicable.

Applicability is resolved first.

51. PLANNING AND SITE ENGINE

The site subsystem evaluates applicable requirements including:

- setbacks
- lot coverage
- building height
- site access
- parking
- land use
- building placement
- easements
- rights-of-way
- site constraints

Site rules may reference the Site Model and Building Model simultaneously.

52. ENERGY ENGINE

The energy subsystem is designed to evaluate applicable requirements associated with:

- envelope
- thermal performance
- fenestration
- air barriers
- insulation
- mechanical systems
- energy targets
- performance paths

The engine architecture must support both:

- prescriptive rules

and:

- performance-based rules

53. ASSEMBLY COMPLIANCE

Assemblies may be evaluated against regulatory requirements.

For example:

- wall assembly
 - ↓
 - layers

↓
material properties
↓
thermal properties
↓
fire properties
↓
acoustic properties
↓
applicable requirements

Assembly compliance results reference the Assembly and affected Elements.

54. OCCUPANCY CLASSIFICATION

Occupancy is a foundational compliance variable.

The engine must support:

project occupancy
building occupancy
level occupancy
space occupancy

The hierarchy is:

SPACE
↓
LEVEL
↓
BUILDING
↓
PROJECT

Classification conflicts must be reported.

55. BUILDING CLASSIFICATION

The engine derives a normalized building classification from:

occupancy
height
area
construction
number_of_levels
fire characteristics
project type

The resulting classification becomes an input to applicable rule selection.

56. RULE DEPENDENCY GRAPH

Rules may depend upon other rules.

Example:

```
occupancy_classification
  ↓
occupant_load
  ↓
required_egress
  ↓
exit_capacity
  ↓
travel_distance
```

The engine must support dependency ordering.

57. RULE EXECUTION GRAPH

The compliance execution graph therefore becomes:

```
CLASSIFICATION
  ↓
CHARACTERISTICS
  ↓
APPLICABILITY
  ↓
DEPENDENT_CALCULATIONS
  ↓
PRIMARY_RULES
  ↓
SECONDARY_RULES
  ↓
DOCUMENTATION_RULES
```

58. DETERMINISTIC EXECUTION

Given identical:

```
model_version
code_manifest
engine_version
rule_set
```

the Compliance Engine should produce the same result.

This is a core reproducibility requirement.

59. COMPLIANCE ENGINE VERSION

Every compliance run records:

```
engine_version  
rule_set_version
```

This permits historical comparison between engine releases.

A change in computational logic must not silently rewrite historical results.

60. RULE VERSIONING

Rules may change over time.

The system therefore treats rule definitions as versioned objects.

Conceptually:

```
RULE 100  
VERSION 1  
VERSION 2  
VERSION 3
```

Each rule version is associated with its source version.

61. RULE HASH

Each normalized rule definition should receive a content hash.

Conceptually:

```
normalized_rule  
  ↓  
SHA-256  
  ↓  
rule_hash
```

This provides deterministic identification of the rule logic used.

62. APPLICABILITY HASH

A resolved applicability set should also be hashable.

Conceptually:

```
project characteristics
+
jurisdiction
+
code manifest
+
rule versions
↓
applicability hash
```

This supports reproducibility and cache validation.

63. COMPLIANCE CACHE

Safe deterministic compliance results may be cached using:

```
project_id
model_version_id
code_manifest_id
engine_version
rule_set_version
```

If any component changes, the cache becomes invalid.

64. STALE RESULT PROTECTION

A compliance result generated for:

```
MODEL VERSION 17
```

must not be presented as current for:

```
MODEL VERSION 18
```

The API must expose the model version associated with every result.

65. COMPLIANCE API

The Compliance Engine shall expose:

```
/api/v1/projects/{project_id}/compliance
```

with resource groups:

```
jurisdictions
sources
```

manifests
rules
runs
results
reports

66. CREATE CODE MANIFEST

Endpoint:

POST /api/v1/projects/{project_id}/compliance/manifests

Request:

```
{
  "jurisdiction": {
    "country": "CA",
    "province_state": "AB",
    "municipality": "Calgary"
  },
  "application_date": "2026-08-10"
}
```

Response:

```
{
  "data": {
    "manifest_id": "uuid",
    "status": "DRAFT"
  }
}
```

67. ACTIVATE CODE MANIFEST

Endpoint:

POST /api/v1/projects/{project_id}/compliance/manifests/{manifest_id}/activate

The activation process verifies:

source integrity
source availability
version metadata
effective dates
rule availability
jurisdiction

The resulting manifest becomes:

ACTIVE

68. FINALIZE CODE MANIFEST

Endpoint:

POST /api/v1/projects/{project_id}/compliance/manifests/{manifest_id}/finalize

The finalized manifest becomes immutable.

The final response contains:

```
manifest_id
content_hash
source_count
rule_count
effective_date
status
```

69. START COMPLIANCE RUN

Endpoint:

POST /api/v1/projects/{project_id}/compliance/runs

Request:

```
{
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "code_manifest_id": "uuid",
  "categories": [
    "BUILDING",
    "FIRE",
    "ACCESSIBILITY",
    "PLANNING",
    "ENERGY"
  ]
}
```

Response:

```
{
  "data": {
    "compliance_run_id": "uuid",
    "status": "QUEUED"
  }
}
```

70. COMPLIANCE RUN EXECUTION

The run executes asynchronously through the established worker architecture.

```
API
↓
Redis
↓
Compliance Worker
↓
Project Model Service
↓
Rule Engine
↓
Geometry Service
↓
Validation Persistence
```

The API remains responsive during expensive analysis.

71. COMPLIANCE JOB

The job payload contains:

```
project_id
revision_id
model_version_id
code_manifest_id
engine_version
rule_set_version
requested_categories
```

The worker verifies that the model version remains valid before execution.

72. COMPLIANCE RESULT API

Endpoint:

```
GET /api/v1/projects/{project_id}/compliance/results
```

Filters:

```
status
severity
category
rule_id
revision_id
model_version_id
```

The API returns only results associated with the requested project context.

73. SINGLE RESULT

Endpoint:

GET /api/v1/projects/{project_id}/compliance/results/{result_id}

The response contains:

result
rule
source
model_version
affected_objects
calculation
provenance

74. COMPLIANCE SUMMARY

A run produces an aggregate summary:

total_rules
applicable_rules
passed
failed
warnings
review_required
not_applicable
undetermined
errors

Example:

```
{
  "total_rules": 412,
  "applicable_rules": 287,
  "passed": 251,
  "failed": 12,
  "warnings": 9,
  "review_required": 11,
  "not_applicable": 118,
  "undetermined": 3,
  "errors": 1
}
```

75. COMPLIANCE REPORT

The Documentation Engine may consume a compliance run and produce:

Compliance Summary
Rule Matrix
Exceptions
Failures
Warnings
Review Items
Source References
Model References

The report remains derived documentation.

The underlying compliance results remain authoritative.

76. DRAWING INTEGRATION

Compliance results may reference drawings.

For example:

```
validation_result
  ↓
affected_object
  ↓
view
  ↓
drawing
  ↓
sheet
```

This allows a reviewer to move from:

regulatory finding

to:

affected architectural element

to:

drawing representation

77. REDLINE INTEGRATION

A compliance finding may generate a redline recommendation.

The chain is:

```
COMPLIANCE RESULT
↓
DESIGN RECOMMENDATION
↓
REDLINE
↓
ARIA COMMAND
↓
MODEL REVISION
↓
REVALIDATION
```

This creates a closed compliance-revision loop.

78. COMPLIANCE REVISION LOOP

The complete iterative process becomes:

```
MODEL
↓
COMPLIANCE
↓
FINDINGS
↓
REDLINES / RECOMMENDATIONS
↓
ARIA
↓
COMMAND
↓
NEW REVISION
↓
NEW MODEL VERSION
↓
COMPLIANCE
```

The system does not overwrite the previous compliance state.

79. PROFESSIONAL REVIEW BOUNDARY

Certain determinations may require professional review.

The engine therefore supports:

```
AUTOMATED
REVIEW_REQUIRED
PROFESSIONAL_REVIEW
ACCEPTED
REJECTED
```

The engine records the result.

It does not impersonate an approving authority.

80. AUTHORITY APPROVAL BOUNDARY

Compliance analysis is distinct from municipal approval.

The architecture therefore distinguishes:

COMPUTATIONAL COMPLIANCE

from:

PROFESSIONAL REVIEW

and:

AUTHORITY APPROVAL

These are separate states in the OneDraft lifecycle.

81. COMPLIANCE CONFIDENCE

The engine may report computational confidence for machine-derived determinations.

However:

confidence != approval

and:

confidence != legal determination

The provenance chain remains the controlling mechanism.

82. HUMAN REVIEW QUEUE

Rules requiring human interpretation enter:

Compliance Review Queue

Each review item contains:

project_id
revision_id

model_version_id
rule_id
finding
source
reason
affected_objects
required_review_type
status

83. REVIEW STATES

Initial review states:

OPEN
ASSIGNED
IN_REVIEW
RESOLVED
REJECTED
DEFERRED

A review resolution becomes part of the project provenance.

84. REVIEW RECORD

A professional review record may include:

reviewer_id
professional_role
scope
result
comments
reviewed_at

This integrates directly with:

revisions.professional_reviews

85. COMPLIANCE AUDIT TRAIL

Every significant compliance event produces an audit event.

Examples:

CODE_SOURCE_IMPORTED
CODE_SOURCE_VERIFIED
MANIFEST_CREATED
MANIFEST_ACTIVATED

MANIFEST_FINALIZED
RULE_RESOLVED
COMPLIANCE_RUN_STARTED
COMPLIANCE_RUN_COMPLETED
COMPLIANCE_RESULT_CREATED
REVIEW_CREATED
REVIEW_RESOLVED

86. AUDIT IMMUTABILITY

Compliance audit events are append-only.

They are not edited to represent later state.

Corrections create new events.

This preserves the chronological record.

87. DATABASE ADDITIONS

The initial persistence layer already contains:

compliance.code_sources
compliance.code_rules
compliance.code_manifests
validation.validation_runs
validation.validation_results
revisions.professional_reviews
audit.project_events

The Compliance Engine may add:

compliance.jurisdictions
compliance.source_artifacts
compliance.rule_versions
compliance.applicability_results
compliance.characteristics
compliance.review_items
compliance.rule_dependencies

88. JURISDICTIONS TABLE

Conceptually:

compliance.jurisdictions

Fields:

```
id UUID PRIMARY KEY
country VARCHAR(128) NOT NULL
province_state VARCHAR(128)
municipality VARCHAR(255)
district VARCHAR(255)
jurisdiction_code VARCHAR(128)
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

89. RULE VERSIONS TABLE

compliance.rule_versions

Fields:

```
id UUID PRIMARY KEY
rule_id UUID NOT NULL
version INTEGER NOT NULL
rule_definition JSONB NOT NULL
rule_hash VARCHAR(128) NOT NULL
effective_date DATE
superseded_date DATE
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(rule_id, version)
```

90. APPLICABILITY RESULTS TABLE

compliance.applicability_results

Fields:

```
id UUID PRIMARY KEY
run_id UUID NOT NULL
rule_id UUID NOT NULL
model_version_id UUID NOT NULL
status VARCHAR(32) NOT NULL
conditions JSONB
explanation TEXT
created_at TIMESTAMPTZ NOT NULL
```

91. CHARACTERISTICS TABLE

compliance.characteristics

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
characteristic_type VARCHAR(128) NOT NULL
value JSONB NOT NULL
unit VARCHAR(32)
source_objects JSONB
calculation JSONB
created_at TIMESTAMPTZ NOT NULL
```

92. RULE DEPENDENCIES TABLE

compliance.rule_dependencies

Fields:

```
id UUID PRIMARY KEY
rule_id UUID NOT NULL
depends_on_rule_id UUID
dependency_type VARCHAR(64) NOT NULL
metadata JSONB
created_at TIMESTAMPTZ NOT NULL
```

This allows construction of the regulatory execution graph.

93. REVIEW ITEMS TABLE

compliance.review_items

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
validation_result_id UUID
rule_id UUID
reason TEXT NOT NULL
required_review_type VARCHAR(128)
assigned_to UUID
status VARCHAR(32) NOT NULL
resolution TEXT
```

created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL

94. COMPLIANCE SERVICE

The Python service boundary becomes:

packages/domain/
compliance/

apps/api/
compliance/

apps/worker/
compliance/

The service should expose domain operations rather than database primitives.

95. CORE COMPLIANCE INTERFACES

Conceptually:

JurisdictionResolver

SourceRegistry

CodeManifestService

RuleRegistry

ApplicabilityEngine

CharacteristicEngine

RuleExecutionEngine

SpatialComplianceEngine

ComplianceRunService

ComplianceReportService

ReviewQueueService

Each service has a defined responsibility.

96. JURISDICTION RESOLVER

`resolve(project)`

returns:

`JurisdictionResolution`

The resolver must preserve the basis for its determination.

97. MANIFEST SERVICE

`create_manifest(project)`
`activate_manifest(manifest_id)`
`finalize_manifest(manifest_id)`
`get_manifest(manifest_id)`

Finalization creates an immutable regulatory snapshot.

98. RULE REGISTRY

`get_rule(rule_id)`
`get_rule_version(rule_id, version)`
`list_applicable_rules(manifest, characteristics)`

The registry provides deterministic rule retrieval.

99. APPLICABILITY ENGINE

`evaluate(rule, project_characteristics)`

returns:

`ApplicabilityResult`

with:

`status`
`conditions`
`explanation`

100. CHARACTERISTIC ENGINE

`derive(model_version)`

returns:

`ProjectCharacteristics`

with provenance.

101. RULE EXECUTION ENGINE

`evaluate(rule, model_snapshot, characteristics)`

returns:

`ComplianceResult`

The engine does not mutate the model.

102. SPATIAL COMPLIANCE SERVICE

`evaluate_spatial(rule, geometry_context)`

requests geometry operations from the Geometry Engine.

The Geometry Engine remains the owner of geometric computation.

103. COMPLIANCE RUN SERVICE

`create_run(...)`
`execute_run(...)`
`get_run(...)`
`summarize_run(...)`

The run service coordinates the entire regulatory analysis.

104. ENGINE ORCHESTRATION

The complete execution architecture becomes:

API
↓
Compliance Run Service
↓
Manifest Service
↓
Rule Registry
↓
Applicability Engine
↓
Characteristic Engine
↓
Rule Execution Engine
↓
Geometry Service
↓
Validation Persistence
↓
Audit Event

105. REDIS WORKER MODEL

Long-running compliance execution uses Redis-backed jobs.

Conceptually:

POST /compliance/runs
↓
QUEUE
↓
COMPLIANCE WORKER
↓
RULE EXECUTION
↓
RESULT PERSISTENCE

The API returns immediately with the run identifier.

106. FAILURE HANDLING

A failed compliance run records:

status = FAILED
error_code
error_message
failed_rule
engine_version
timestamp

The system must distinguish:

rule failure

from:

engine failure

and:

source failure

107. SOURCE FAILURE

If a required regulatory source cannot be verified, the system must not fabricate a rule set.

The affected manifest or analysis becomes:

REVIEW_REQUIRED

or:

INVALID

according to the failure condition.

108. MISSING DATA

If the model lacks a characteristic required for a rule:

MISSING_INPUT

is recorded.

The resulting compliance status may become:

UNDETERMINED

rather than automatically passing or failing.

109. MODEL-CONFLICT DETECTION

The Compliance Engine should identify contradictory model states.

Examples:

space occupancy conflicts with building occupancy
level elevation conflicts with geometry

element classification conflicts with assembly
door geometry conflicts with element parameters

These become:

COORDINATION

or:

ERROR

results depending upon severity.

110. COMPLIANCE DATA QUALITY

Before rule execution, the engine performs a data-quality pass.

Conceptually:

MODEL
↓
DATA QUALITY
↓
CLASSIFICATION
↓
CHARACTERISTICS
↓
RULE ANALYSIS

This prevents invalid model data from being mistaken for regulatory noncompliance.

111. DOCUMENTATION COMPLIANCE

The engine may evaluate whether required documentation exists.

Examples:

required drawing
required schedule
required note
required dimension
required annotation
required detail

This creates a bridge between regulatory requirements and the Documentation Engine.

112. DRAWING COORDINATION

The Compliance Engine may detect when documentation does not correspond to the model.

For example:

MODEL:
window exists

DRAWING:
window absent

The finding is a coordination/documentation result rather than a geometric model mutation.

113. CODE-TO-MODEL TRACEABILITY

Each rule must be able to identify its affected model objects.

Example:

RULE
↓
building
↓
level
↓
space
↓
door

This allows the user to navigate from a compliance finding directly to the responsible model object.

114. MODEL-TO-CODE TRACEABILITY

The reverse relationship must also exist.

Given:

Element_ID

the system should be able to determine:

Which compliance rules evaluate this element?
Which results were produced?
Which code sources support those rules?

This is essential for debugging and professional review.

115. COMPLIANCE GRAPH

The regulatory graph becomes:

```
CODE SOURCE
  ↓
CODE RULE
  ↓
APPLICABILITY
  ↓
CHARACTERISTIC
  ↓
MODEL OBJECT
  ↓
VALIDATION RESULT
  ↓
REVIEW / RECOMMENDATION
```

This graph forms a regulatory dependency layer above the computational Project Model.

116. COMPLIANCE SNAPSHOT

A finalized compliance state is identified by:

```
project_id
revision_id
model_version_id
code_manifest_id
engine_version
rule_set_version
```

This tuple uniquely identifies the regulatory computational context.

117. RECONSTRUCTION REQUIREMENT

Given:

```
Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
Compliance_Engine_Version
```

OneDraft must be capable of reconstructing the compliance run and its underlying determinations.

118. REPRODUCIBILITY TEST

The Compliance Engine must eventually pass:

```
MODEL VERSION 17
+
CODE MANIFEST 4
+
ENGINE VERSION 0.1
+
RULE SET VERSION 3
```

producing the same deterministic results when all source inputs remain unchanged.

119. REGULATORY UPDATE TEST

When a regulatory source changes:

```
CODE MANIFEST 4
  ↓
CODE MANIFEST 5
```

the historical analysis remains attached to Manifest 4.

A new analysis may be executed against Manifest 5.

The system therefore preserves both:

historical compliance state

and:

current compliance state

120. MODEL UPDATE TEST

When the architectural model changes:

```
MODEL VERSION 17
  ↓
MODEL VERSION 18
```

the Compliance Engine must invalidate current-state compliance for Version 17 while retaining its historical record.

A new run is created for Version 18.

121. ARIA ITERATION TEST

The complete AI compliance iteration becomes:

```
MODEL
↓
COMPLIANCE
↓
FAILURE
↓
ARIA
↓
DESIGN PROPOSAL
↓
COMMAND
↓
MODEL VERSION +1
↓
COMPLIANCE
```

The system continues until the project reaches the desired review state.

122. NO DIRECT AI REGULATORY AUTHORITY

Aria may interpret structured compliance findings and assist with design modifications.

Aria does not become the authority determining whether a regulatory source exists or whether a professional or municipal authority has approved a project.

The regulatory engine remains source-driven and provenance-driven.

123. COMPLIANCE ENGINE SECURITY

Compliance resources are subject to:

```
organization membership
project membership
authorization scopes
```

Initial scopes include:

```
compliance:read
compliance:execute
admin:project
```

Source administration may require:

admin:organization

124. TENANCY ISOLATION

All compliance queries must be scoped by:

organization_id
project_id

A user must never obtain regulatory project data from another organization merely by possessing a UUID.

Authorization occurs before resource access.

125. IDEMPOTENCY

Mutating compliance endpoints support:

Idempotency-Key

This applies particularly to:

manifest creation
manifest activation
manifest finalization
compliance run creation
review creation

126. API ERROR MODEL

Initial errors include:

JURISDICTION_UNRESOLVED
SOURCE_UNAVAILABLE
SOURCE_UNVERIFIED
MANIFEST_INVALID
MANIFEST_FINALIZED
RULE_NOT_FOUND
RULE_VERSION_NOT_FOUND
APPLICABILITY_UNDETERMINED
MODEL_INPUT_MISSING
MODEL_VERSION_STALE
COMPLIANCE_RUN_FAILED
COMPLIANCE_RUN_STALE
REVIEW_REQUIRED
AUTHORIZATION_FAILED

127. OBSERVABILITY

The Compliance Engine must emit structured telemetry.

Minimum fields:

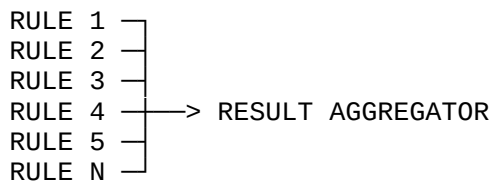
```
request_id
project_id
run_id
model_version_id
code_manifest_id
rule_id
engine_version
duration_ms
status
```

This enables performance and correctness monitoring.

128. PERFORMANCE TARGET

The engine should support parallel evaluation of independent rules.

Conceptually:



Rules with dependencies must execute according to the dependency graph.

129. RULE EXECUTION CACHE

Deterministic rule calculations may be cached.

Cache keys must include:

```
rule_version
model_version
characteristic_hash
geometry_hash
```

A stale cache entry must never become authoritative.

130. GEOMETRY HASH INTEGRATION

Because the Geometry Engine already produces deterministic geometry hashes, compliance spatial checks may use:

`geometry_hash`

to detect whether the spatial inputs have changed.

This avoids unnecessary recomputation.

131. DOCUMENT HASH INTEGRATION

Compliance reports and final document packages may be hashed.

The provenance chain becomes:

```
MODEL HASH
↓
GEOMETRY HASH
↓
COMPLIANCE HASH
↓
DRAWING HASH
↓
DOCUMENT HASH
```

This provides an integrity-oriented project history.

132. COMPLIANCE HASH

A compliance run may generate a deterministic hash from:

```
model_version_id
code_manifest_id
engine_version
rule_set_version
result set
```

This hash becomes the identity of the computational compliance state.

133. FINAL DOCUMENT PROVENANCE

A final OneDraft package may therefore identify:

Project
Revision
Model Version
Code Manifest
Compliance Run
Drawing Set
Document Package
Acceptance

The regulatory basis is preserved with the deliverable.

134. COMPLIANCE REPORTING LAYER

The Documentation Engine consumes structured results and generates:

Compliance Matrix
Rule Summary
Exception Matrix
Outstanding Review Items
Source Register
Model Traceability

The report itself is derived.

The database remains the source of compliance state.

135. FRONTEND COMPLIANCE INTERFACE

The frontend shall expose a Compliance workspace.

Conceptually:

PROJECT

- Model
- Drawings
- Redlines
- Compliance
 - Summary
 - Findings
 - Rules
 - Sources
 - Review
- Documents

136. COMPLIANCE DASHBOARD

The dashboard should expose:

Compliance Status
Applicable Rules
Passed
Failed
Warnings
Review Required
Undetermined
Source Status
Current Manifest
Model Version
Last Run

The dashboard is a presentation of stored computational state.

137. FINDING NAVIGATION

Selecting a finding should permit navigation:

Finding
↓
Rule
↓
Source
↓
Affected Object
↓
Model
↓
Drawing

This becomes one of the primary user-facing advantages of the OneDraft architecture.

138. COMPLIANCE REVIEW WORKFLOW

The intended workflow is:

1. Create Project
2. Resolve Jurisdiction
3. Build Code Manifest
4. Verify Regulatory Sources
5. Finalize Manifest

6. Build Project Model
 7. Derive Characteristics
 8. Resolve Applicable Rules
 9. Run Compliance
 10. Review Findings
 11. Generate Recommendations
 12. Apply Approved Model Changes
 13. Create New Revision
 14. Re-run Compliance
 15. Generate Documentation
 16. Professional Review
 17. Customer Acceptance
-

139. COMPLIANCE ENGINE TEST SUITE

The first test suite shall contain:

jurisdiction resolution
source registration
source hashing
manifest creation
manifest immutability
rule registration
rule versioning
applicability evaluation
unit conversion
threshold evaluation
spatial evaluation
characteristic derivation
rule dependencies
compliance run creation
compliance result persistence
stale model detection
stale manifest detection
review queue creation
audit event creation

140. FIRST REGULATORY INTEGRATION TEST

The first complete regulatory integration test should execute:

1. Create organization
2. Create user
3. Create project
4. Define jurisdiction
5. Register regulatory source
6. Create source version
7. Create Code Manifest
8. Finalize Code Manifest
9. Create building
10. Create levels
11. Create spaces
12. Create walls
13. Create doors
14. Create windows
15. Derive project characteristics
16. Resolve applicable rules
17. Execute compliance run
18. Produce validation results
19. Identify failures
20. Create review items
21. Create Aria design proposal
22. Execute approved command
23. Create new model revision
24. Re-run compliance
25. Verify historical results remain unchanged
26. Generate compliance report

- 27. Attach compliance provenance to document package
 - 28. Retrieve complete regulatory history
-

141. MVP COMPLIANCE SUCCESS CRITERIA

The Compliance Engine is successful when it can:

Resolve a project jurisdiction

Register regulatory sources

Preserve source provenance

Create an immutable Code Manifest

Represent regulatory requirements as structured rules

Determine rule applicability

Derive model characteristics

Evaluate deterministic requirements

Perform spatial compliance checks

Record validation results

Identify affected model objects

Create review items

Preserve historical compliance runs

Detect stale model versions

Detect stale regulatory manifests

Generate compliance summaries

Integrate findings with Aria

Integrate findings with revisions

Integrate findings with documentation

Reconstruct historical compliance state

142. API SUCCESS CRITERIA

The API implementation is successful when a client can execute:

```
CREATE PROJECT
↓
RESOLVE JURISDICTION
↓
CREATE CODE MANIFEST
↓
FINALIZE MANIFEST
↓
CREATE MODEL
↓
DERIVE CHARACTERISTICS
↓
RESOLVE RULES
↓
RUN COMPLIANCE
↓
RETRIEVE FINDINGS
↓
CREATE REVIEW ITEM
↓
PROPOSE MODEL CHANGE
↓
CREATE NEW REVISION
↓
RE-RUN COMPLIANCE
↓
GENERATE REPORT
```

without directly accessing PostgreSQL.

143. ENGINE SUCCESS CRITERIA

The Compliance Engine must satisfy four fundamental properties:

DETERMINISTIC

The same inputs produce the same computational result.

TRACEABLE

Every result can be traced to its rule and source.

VERSIONED

Historical regulatory determinations remain reproducible.

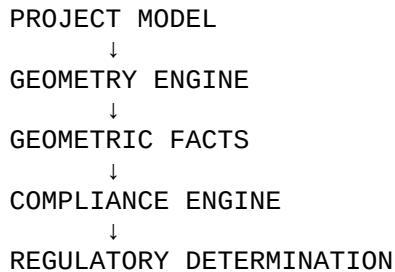
NON-DESTRUCTIVE

Compliance analysis never silently modifies the Project Model.

144. ARCHITECTURAL RELATIONSHIP TO GEOMETRY

The Compliance Engine does not replace the Geometry Engine.

The relationship is:



The Geometry Engine answers:

What is geometrically true?

The Compliance Engine answers:

Does the model satisfy the applicable rule?

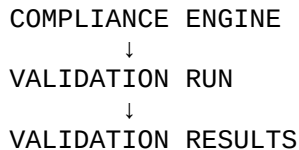
This separation is fundamental.

145. ARCHITECTURAL RELATIONSHIP TO VALIDATION

The Compliance Engine produces regulatory validation inputs and results.

The Validation subsystem provides the common persistence and result framework.

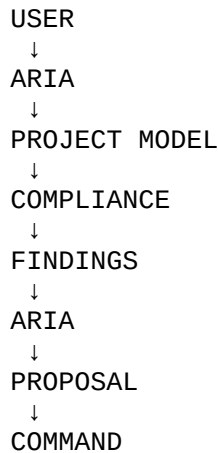
The relationship is:



Other validation domains may use the same infrastructure.

146. ARCHITECTURAL RELATIONSHIP TO ARIA

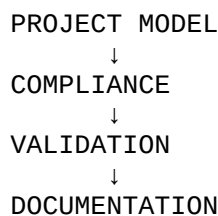
Aria operates above the Compliance Engine.



Aria is therefore the reasoning/orchestration layer, while Compliance remains a deterministic domain subsystem.

147. ARCHITECTURAL RELATIONSHIP TO DOCUMENTATION

Documentation consumes compliance state.



The document package records the regulatory context from which it was generated.

148. ARCHITECTURAL RELATIONSHIP TO CUSTOMER ACCEPTANCE

Customer acceptance occurs after the system has produced the appropriate project state and documentation.

Conceptually:

MODEL



COMPLIANCE



VALIDATION



DOCUMENTATION



PROFESSIONAL REVIEW



CUSTOMER ACCEPTANCE

Acceptance does not alter the underlying historical compliance record.

149. ONE-DRAFT REGULATORY SYSTEM OF RECORD

For regulatory state, the hierarchy is:

1. Code Source
2. Code Source Version
3. Code Rule
4. Code Manifest
5. Project Model Version
6. Applicability Determination
7. Compliance Run
8. Validation Result
9. Professional Review
10. Document Package

This preserves the complete regulatory provenance chain.

150. COMPLIANCE RECONSTRUCTION TEST

OneDraft must eventually pass the following test:

Given:

Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
Compliance_Run_ID

the system must reconstruct:

jurisdiction
regulatory sources
source versions
rules
applicability
project characteristics
model inputs
rule evaluations
validation results
review items
compliance summary

without relying upon undocumented external state.

151. INITIAL IMPLEMENTATION ARTIFACTS

The Compliance Engine implementation package shall consist initially of:

```
compliance/  
  jurisdiction_resolver.py  
  source_registry.py  
  manifest_service.py  
  rule_registry.py  
  applicability_engine.py  
  characteristic_engine.py  
  rule_execution_engine.py  
  spatial_compliance.py  
  compliance_run_service.py  
  review_service.py
```

Database migration additions:

026_compliance_engine.sql

API specification additions:

/docs/openapi/compliance.yaml

Domain models:

packages/domain/compliance/

Worker:

apps/worker/compliance_worker.py

152. FIRST IMPLEMENTATION MIGRATION

The next PostgreSQL migration shall establish the additional regulatory tables:

```
compliance.jurisdictions
compliance.rule_versions
compliance.applicability_results
compliance.characteristics
compliance.rule_dependencies
compliance.review_items
```

The migration shall preserve the existing:

```
compliance.code_sources
compliance.code_rules
compliance.code_manifests
validation.validation_runs
validation.validation_results
```

rather than replacing them.

153. FIRST OPENAPI IMPLEMENTATION

The OpenAPI contract shall add:

```
GET    /projects/{project_id}/compliance/manifests
POST   /projects/{project_id}/compliance/manifests

GET    /projects/{project_id}/compliance/manifests/{manifest_id}
POST   /projects/{project_id}/compliance/manifests/{manifest_id}/activate
POST   /projects/{project_id}/compliance/manifests/{manifest_id}/finalize

GET    /projects/{project_id}/compliance/runs
POST   /projects/{project_id}/compliance/runs

GET    /projects/{project_id}/compliance/runs/{run_id}

GET    /projects/{project_id}/compliance/results
GET    /projects/{project_id}/compliance/results/{result_id}

GET    /projects/{project_id}/compliance/reviews
POST   /projects/{project_id}/compliance/reviews
```

154. FIRST DOMAIN TYPES

The Python domain layer shall initially define:

```
Jurisdiction
JurisdictionResolution
CodeSource
CodeSourceVersion
CodeManifest
CodeRule
CodeRuleVersion
ApplicabilityResult
ProjectCharacteristic
ComplianceRun
ComplianceResult
ComplianceReviewItem
ComplianceSummary
```

All domain objects use UUID identifiers and explicit version/provenance references where required.

155. FIRST WORKER

The first worker implementation shall execute:

```
ComplianceRunJob
```

with:

```
project_id
revision_id
model_version_id
code_manifest_id
```

The worker shall:

```
load model snapshot
load manifest
resolve rules
derive characteristics
execute applicable rules
persist results
persist summary
write audit event
complete run
```

156. FIRST REGRESSION TEST

The first compliance regression test shall establish:

Project
+
Known Model Version
+
Known Code Manifest
+
Known Rule Set
=
Known Compliance Results

The expected results become version-controlled test fixtures.

157. STACK POSITION

The OneDraft stack now contains:

```
PRODUCT DEFINITION
  ↓
SYSTEM ARCHITECTURE
  ↓
APPLICATION STACK
  ↓
PERSISTENCE / API
  ↓
COMPUTATIONAL PROJECT MODEL
  ↓
GEOMETRY / PARAMETRIC ENGINE
  ↓
COMPLIANCE / REGULATORY ENGINE
```

The next layers will build upon this regulatory foundation.

158. COMPLETE COMPUTATIONAL CHAIN

OneDraft now has the following fundamental computational chain:

```
USER SPECIFICATION
  ↓
ARIA
  ↓
PROJECT MODEL
  ↓
PARAMETRIC GEOMETRY
  ↓
REGULATORY ANALYSIS
  ↓
VALIDATION
  ↓
DRAWING GENERATION
  ↓
REDLINE
```

↓
ARIA REVISION
↓
NEW MODEL VERSION
↓
REVALIDATION
↓
FINAL DOCUMENT PACKAGE
↓
PROFESSIONAL REVIEW
↓
CUSTOMER ACCEPTANCE

159. FUNDAMENTAL ARCHITECTURAL RULE

The Compliance Engine is not a separate document-checking utility.

It is a computational subsystem operating directly against the versioned Project Model.

Therefore:

CODE
+
MODEL
+
GEOMETRY
+
PROVENANCE

produce:

COMPLIANCE STATE

That compliance state then becomes an input to the remaining OneDraft production pipeline.

160. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now has:

Product definition

System architecture

Technology architecture

Executable application stack

Persistence model

API contract

Computational Project Model

Geometry and parametric model engine

Regulatory source model

Jurisdiction resolution

Code Manifest

Code Rule model

Applicability engine

Project characteristic engine

Deterministic rule execution

Spatial compliance

Validation integration

Compliance provenance

Professional review boundary

Aria compliance interface

Revision integration

Documentation integration

Regulatory reconstruction

Compliance regression framework

The Compliance & Regulatory Engine is therefore established as the next executable OneDraft subsystem.

161. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package shall consist of:

Artifact A

`026_compliance_engine.sql`

The PostgreSQL migration establishing the regulatory execution tables.

Artifact B

`compliance.yaml`

The OpenAPI definition for jurisdiction, manifests, rules, compliance runs, results, and review resources.

Artifact C

`compliance_types.py`

The Pydantic/domain representations for regulatory objects and compliance state.

Artifact D

`compliance_engine.py`

The first executable regulatory analysis service.

Artifact E

`compliance_worker.py`

The asynchronous worker responsible for executing compliance runs.

Artifact F

`test_compliance_engine.py`

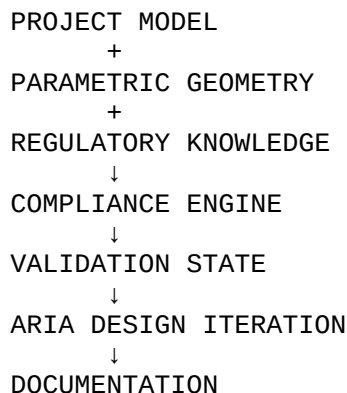
The first deterministic compliance integration and regression test suite.

162. FINAL ARCHITECTURAL DECISION

OneDraft shall not treat regulatory compliance as a final-stage checklist applied to completed drawings.

Compliance is a computational layer operating continuously against the Project Model.

The governing architecture is therefore:



The architectural model remains the authoritative computational representation of the project.

The Code Manifest remains the authoritative regulatory snapshot used for a particular compliance determination.

The Compliance Engine evaluates the relationship between those two states.

The resulting validation state is versioned, traceable, reproducible, and retained.

163. SPECIFICATION STATUS

ONEDRAFT COMPLIANCE & REGULATORY ENGINE v0.1

STATUS: ESTABLISHED

The regulatory layer is now defined.

The next implementation stage is the conversion of this specification into executable artifacts:

```
026_compliance_engine.sql  
compliance.yaml  
compliance_types.py  
compliance_engine.py  
compliance_worker.py  
test_compliance_engine.py
```